

SENSEI

Smart Environmental Navigation System for Enhanced Independence

Complete Project Learning Guide

Version 1.0.0 | React Native (Expo) + Node.js/Express + MongoDB
Logics, Algorithms, Technologies & Module-by-Module Architecture

Table of Contents

1. Project Purpose & Design Philosophy

2. Technology Stack

3. System Architecture Overview

4. Core Algorithms & Logic

5. Application Entry & Navigation Flow

6. Mobile App — Module by Module

7. Screens — Module by Module

8. Components & Utilities

9. Backend Server — Module by Module

10. Data Flow Diagrams

11. Configuration & Deployment

12. Testing Strategy

13. Learning Roadmap (What to Study)

1. Project Purpose & Design Philosophy

SENSEI is a mobile accessibility application built for visually impaired users. It combines computer vision, GPS navigation, voice control, spatial audio, and emergency safety into one hands-free experience.

Core design goals:

- **Hands-free operation** — Wake word ("Hey Sensei") + 100+ voice command variations
- **Real-world obstacle awareness** — On-device object detection (COCO-SSD) with hazard scoring
- **Ground safety** — Depth grid analysis for stairs, drop-offs, uneven surfaces
- **Navigation** — Outdoor (Google Maps) + indoor (graph/Dijkstra) + pinned locations
- **Hybrid cloud/offline** — Heavy AI can run on-device; OCR and logging use LAN server when available
- **Multi-modal feedback** — TTS, spatial audio, vibration, on-screen AR overlays

The app is formed as a **service-oriented React Native architecture**: each capability (detection, navigation, voice, emergency) lives in a singleton service class. Screens orchestrate services; services rarely depend on UI. The backend stores user data, sessions, and serves ML model weights + server-side OCR.

2. Technology Stack

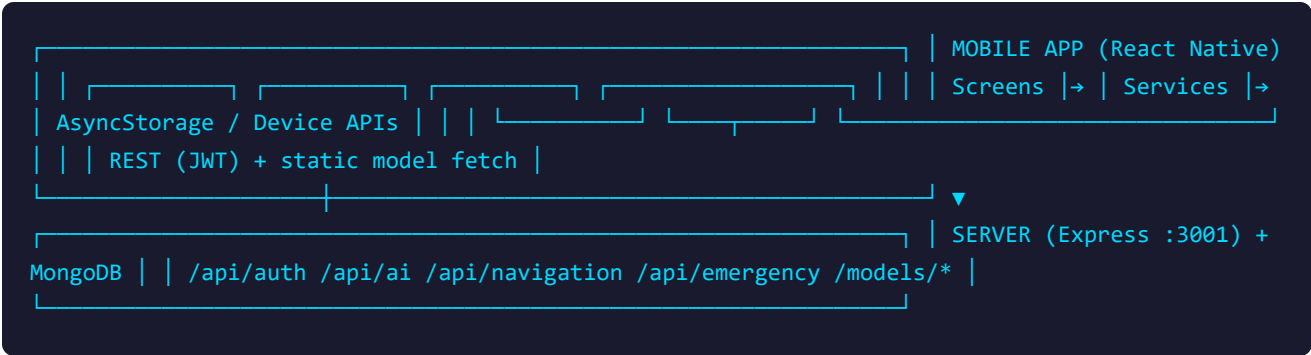
2.1 Mobile Client

Technology	Role in SENSEI
React Native 0.81 + Expo SDK 54	Cross-platform mobile framework; dev client for native modules
React Navigation 6	Tab + stack navigation (Home, Navigation, AR, Settings)
TensorFlow.js + tfjs-react-native	On-device COCO-SSD object detection (WebGL/CPU backend)
@tensorflow-models/coco-ssd	Pre-trained SSD MobileNet models (80 COCO classes)
@vladmandic/face-api	TinyFaceDetector + FaceExpressionNet for emotion screen
expo-camera	Camera preview + snapshot capture for detection pipeline
react-native-vision-camera + vision-camera-object-detector	Optional real-time ML Kit detection path (dev build only)
Three.js + expo-gl + expo-three	3D AR scene rendering (ARService, ARCanvas)
expo-location	GPS position, watch, haversine distance
react-native-maps	Map display, polylines, markers in NavigationScreen
expo-speech	Text-to-speech output queue with priority
expo-speech-recognition	Continuous speech recognition for voice commands
expo-audio + react-native-fs	Spatial audio playback; WAV cache for mic tones
AsyncStorage	JWT token, settings, indoor buildings, pinned locations, routes
expo-sensors (Accelerometer)	Fall detection in EmergencyService
expo-contacts / expo-sms / expo-linking	Emergency contact import, SMS, WhatsApp location share

2.2 Backend Server

Technology	Role in SENSEI
Node.js + Express 4	REST API on port 3001
MongoDB + Mongoose 8	Users, sessions, detection logs, routes, emergency alerts
JWT + bcryptjs	Authentication (7-day token expiry)
Tesseract.js	Server-side OCR from base64 JPEG images
Socket.IO	Real-time emergency event broadcast
express.static	Serves /models/face-api and /models/coco-ssd for LAN model loading

3. System Architecture Overview



Formation pattern: The project grew from a navigation prototype into a full assistive stack. Each new capability was added as an isolated service with a singleton export, wired through `AppInitializer` at boot and through screen-specific effects (especially `ARScreen` for the vision pipeline).

4. Core Algorithms & Logic

4.1 Object Detection Pipeline (COCO-SSD)

File: `ObjectDetectionService.js`

Model: COCO-SSD (Single Shot MultiBox Detector + MobileNet backbone). Detects 80 object classes (person, car, chair, etc.).

Load sequence

1. GPU compatibility probe (Samsung Exynos → unknown; MediaTek PowerVR → unsafe/CPU-only)
2. TensorFlow backend selection: `rn-webgl` → `webgl` → `cpu`, with `matMul` + `depthwiseConv2d` validation
3. Backend result cached in `AsyncStorage` to skip validation on next cold start
4. Model load order: device RNFS cache → LAN server (if weights verified) → Google CDN
5. Non-blocking warmup inference to compile GPU shaders

Per-frame detection (`detectFromTensor`)

1. **Rate limiting** — Target 24 FPS (GPU) or 4 FPS (CPU); skip if previous frame still processing
2. **Tensor prep** — Cast to int32 RGB 0–255 as required by coco-ssd
3. **model.detect(tensor, maxBoxes, minScore)** — Raw SSD inference
4. **Post-filter** — Clamp bbox to image bounds; reject tiny boxes (area thresholds differ for large vs small objects)
5. **False-positive class filter** — Food/sports classes require higher confidence (≥ 0.60)
6. **NMS (Non-Max Suppression)** — Per-class greedy NMS in JS (≤ 8 boxes) or TF async NMS for larger sets. Removes overlapping duplicate boxes.
7. **Distance estimation** — Pinhole camera model: $distance = (canonicalHeight \times focalLength) / bboxHeightPixels$. Fallback: $6 / \sqrt{bboxArea}$
8. **Multi-object tracking** — IoU-based association across frames; exponential smoothing on bbox/score/distance
9. **Temporal confirmation** — Detections below bypass threshold (0.55) need 2 consecutive frames (`minConfirmFrames`)
10. **Hazard enrichment** — `HazardScoringService` assigns critical/high/medium/low

Key formulas

```
focalLength = (frameHeight / 2) / tan(vFOV / 2)
distance     = canonicalHeight × focalLength / bboxHeightPixels
IoU(A,B)     = intersectionArea / unionArea
smoothedBox  =  $\alpha \times \text{newBox} + (1-\alpha) \times \text{oldBox}$  ( $\alpha$  = smoothingFactor, default 0.72)
```

4.2 Hazard Scoring

File: HazardScoringService.js

```
score = baseClassWeight + distanceFactor + sizeFactor + positionFactor

baseClassWeight: car/bus/truck=5, person/dog=3, chair=1, etc.
distanceFactor: 6 - min(5.9, log2(1 + 8/dist) × 2) → closer = higher
sizeFactor:     min(5, log10(1 + area×100) × 4)
positionFactor: center=3, left/right=2

Levels: score ≥8 → critical, ≥6 → high, ≥4 → medium, else low
```

4.3 Depth Estimation (7×5 Ground Grid)

File: DepthEstimationService.js

On-device depth without MiDaS neural network — uses geometry + detection fusion.

- Camera modeled at ~1 m height; horizon at normalized Y = 0.42
- Ground distance from bottom-of-frame: $\text{depth} \approx \text{GROUND_K} / (1 - \text{normY})$ where GROUND_K = 0.55
- 7 columns × 5 rows grid maps walkable zones: critical (<0.5 m), near (<1.2 m), mid (<3 m)
- COCO object heights (person=1.7 m, car=1.45 m) refine per-cell depth via bbox height
- **Stair detection from brightness:** ARScreen extracts per-row mean brightness from bottom 60% of frame; horizontal edges (step risers) detected when row-to-row brightness jumps exceed threshold
- Drop-off detection when depth delta between rows exceeds DROP_THRESHOLD (0.45 m)

4.4 Foot Placement & Safety Guidance

File: `FootPlacementService.js`

- **Grid-based guidance (primary):** Scans 7×5 depth grid columns [left|center|right]; finds safest walk direction
- **Object-based fallback:** Uses COCO detections in lower half of frame as ground obstacles
- **Stair temporal filter:** Requires 3 consecutive frames before announcing stairs (reduces texture false positives)
- **Cooldown deduplication:** Separate TTS cooldowns for guidance (2.5 s), drops (2.5 s), stairs (3 s), surface (4 s)
- **Outputs:** Vibration patterns by hazard level + spatial audio beeps + TTS messages

4.5 Voice Command Recognition

File: `VoiceCommandService.js` (~1,900 lines)

- **Wake word:** Continuous listening via expo-speech-recognition; "Hey Sensei" activates command mode
- **Fuzzy matching:** Levenshtein distance + similarity score (0–1); keyword containment with 0.8 word similarity threshold
- **100+ command variations** mapped to actions: navigation, AR start/stop, OCR, emergency, indoor nav, pinned locations, emotion detection
- **Command categories:** app navigation, AR control, object queries, GPS navigation, indoor navigation, emergency, settings
- Commands logged to server `POST /api/ai/voice-command` when authenticated

```
Levenshtein(a,b): dynamic programming edit distance  
similarity(a,b) = 1 - distance / max(len(a), len(b))
```


4.6 Outdoor Navigation

Files: `GoogleMapsService.js` , `NavigationService.js`

- **Route source:** Google Directions API (walking mode) with polyline decoding
- **Fallback geocoding:** Nominatim/OpenStreetMap when Google API key missing
- **Step tracking:** Compare user GPS to next maneuver point; advance step when within threshold (10 m)
- **Guidance:** TTS speaks turn instructions; `SpatialAudioService` plays directional cues
- **Haversine formula** for distance between GPS coordinates
- **Bearing calculation** for heading-relative turn instructions

4.7 Indoor Navigation (Graph + Dijkstra)

File: `IndoorNavigationService.js`

Buildings modeled as graphs (not pixel grids) — voice-definable landmarks.

```
Building → floors → { nodes, edges }  
Node: id, name, type, lat/lng, floor, connectsFloors (stairs/elevator)  
Edge: from, to, distanceM, heading°, instruction text
```

```
Pathfinding: Dijkstra's algorithm on weighted edges (distanceM)  
Multi-floor: routes traverse staircase/elevator nodes  
Localization: nearest node to GPS OR voice "I am at [landmark]"  
Storage: AsyncStorage key @sensei_indoor_buildings
```

4.8 AR Screen Capture Pipeline

File: ARScreen.js

1. `takePictureAsync` snapshot (quality 0.55, no base64 in capture)
2. `ImageManipulator` native resize to 320 px width + EXIF orientation fix
3. `decodeJpeg` (tfjs-react-native) → RGB tensor
4. Optional `resizeBilinear` if dimensions off by >2 px
5. `ObjectDetectionService.detectFromTensor`
6. `extractImageEdgeSignal` — row brightness means for stair detection
7. Depth + FootPlacement every other frame (performance optimization)
8. **Pipelining:** Capture continues while inference runs; latest-frame-wins via `pendingPhotoRef`

4.9 Emotion Detection

File: EmotionDetectionService.js

- `TinyFaceDetector` locates face in JPEG (base64 from front camera)
- `FaceExpressionNet` classifies: happy, sad, angry, fearful, disgusted, surprised, neutral
- Models loaded from LAN `/models/face-api/` via `fetch` + `TF.js`
- `face-api` environment manually configured for React Native (no DOM)

4.10 Emergency & Fall Detection

File: EmergencyService.js

- Accelerometer magnitude spike detection for falls
- Manual SOS triggers alert to server + contacts via SMS/WhatsApp
- Live location broadcast during emergency
- `Socket.IO` on server for real-time alert propagation

5. Application Entry & Navigation Flow

```
index.js → registerRootComponent(App) App.js: └ ApiService.initialize() → restore JWT from
AsyncStorage └ [no token] → AuthScreen (login/register) └ [authenticated] →
NavigationContainer └ RootStack └ MainTabs (Home | Navigation | AR | Settings) └
EmotionDetection (modal) └ VoiceCommandService.startContinuousListening() └ Floating mic
status indicator AppInitializer (called after auth): └ ObjectDetectionService.loadModel()
[parallel preload] └ OCRService / EmotionDetection / DepthEstimation init └
OfflineModeService.pingServer() → setUseCloud(true/false) └
LocationService.requestPermissions() └ SpatialAudioService + BluetoothService init
```

6. Mobile App — Services Module by Module

ApiService.js

Formed as: Central HTTP gateway for all backend communication.

Logic: Stores JWT in AsyncStorage; attaches Bearer header; clears token on 401. Methods for auth, profile, emergency contacts, navigation sessions, OCR, logs, offline maps.

AppInitializer.js

Formed as: Single boot orchestrator called once after login.

Logic: Parallel model preload + sequential service init; announces online/offline via TTS based on server ping.

ObjectDetectionService.js

Formed as: Core vision engine — largest and most complex service (~1,300 lines).

Logic: Full COCO-SSD pipeline described in Section 4.1. Integrates HazardScoring, voice announcements with per-class cooldowns, server session logging.

DepthEstimationService.js

Formed as: Lightweight on-device alternative to server MiDaS endpoint.

Logic: 7×5 grid depth map from perspective geometry + COCO heights + brightness edge analysis. See Section 4.3.

FootPlacementService.js

Formed as: Safety layer on top of depth + detections for blind navigation.

Logic: Grid/object guidance, stair/drop/surface warnings with temporal filters. See Section 4.4.

HazardScoringService.js

Formed as: Pure scoring function — no side effects, easy to test.

Logic: Weighted sum of class, distance, size, position → 0–10 score → level. See Section 4.2.

VoiceCommandService.js

Formed as: App-wide voice OS — grew to 100+ command patterns.

Logic: Wake word → fuzzy match → action dispatch → navigation ref / service callbacks. See Section 4.5.

GoogleMapsService.js + NavigationService.js

Formed as: Split between API calls (GoogleMaps) and session orchestration (Navigation).

Logic: Route fetch, polyline decode, step tracking, TTS guidance. See Section 4.6.

IndoorNavigationService.js

Formed as: Voice-first indoor alternative when GPS is too coarse.

Logic: Graph storage, Dijkstra pathfinding, landmark CRUD via voice. See Section 4.7.

EmotionDetectionService.js

Formed as: On-device face-api wrapper to avoid server round-trip for emotions.

Logic: TF.js init → load TinyFaceDetector + FaceExpressionNet → detectFromBase64. See Section 4.9.

OCRService.js

Formed as: Server-dependent text reading (signs, labels).

Logic: Health check → POST base64 JPEG to `/api/ai/ocr` → Tesseract.js on server → confidence-filtered text.

TextToSpeechService.js

Formed as: Central speech output queue to prevent overlapping announcements.

Logic: Priority queue (critical interrupts normal); deduplication; Android callback watchdog.

SpatialAudioService.js

Formed as: 3D audio feedback for obstacle direction and navigation.

Logic: Pan/volume by angle and distance; directional beeps for hazards.

EmergencyService.js

Formed as: Safety-critical module with local + server sync.

Logic: Fall detection, SOS, contact notification, location sharing. See Section 4.10.

OfflineModeService.js

Formed as: Connectivity awareness layer.

Logic: 5 s health ping; toggles cloud mode; caches routes for offline replay; syncs pending data when back online.

Other Services (summary)

Service	Purpose
ARService.js	Three.js scene, camera, lighting, 3D anchor management
ARNavigationService.js	In-memory AR navigation anchor store + raycast stub
LocationService.js	Expo Location wrapper: permissions, watch, haversine, bearing
SettingsService.js	Persisted detection/navigation prefs with pub/sub listeners
PinnedLocationService.js	Named GPS/indoor pins with fuzzy synonym matching
RouteMemoryService.js	Record/replay routes, recent searches, frequent destinations
BluetoothService.js	BLE scan/connect (mock devices; TODO real BLE)
WearablesService.js	Haptic patterns via Bluetooth layer
MicToneService.js	660/440 Hz WAV beeps for mic on/off feedback

7. Screens — Module by Module

Screen	Lines	Role & How It Was Formed
AuthScreen.js	~200	Gate before app access. Email/password via ApiService. TTS confirms login. Social auth placeholders for future.
HomeScreen.js	~300	Dashboard hub: quick actions (Navigate, AR, Detection), GPS coords, nearby objects summary, system status cards.
NavigationScreen.js	~1,200	Largest screen. MapView, destination search, indoor nav UI, pinned locations, route memory, polyline rendering, turn panel, voice integration. Grew iteratively as navigation features were added.
ARScreen.js	~1,430	Core vision screen. Camera + detection overlays + depth/foot pipeline + OCR + spatial audio. Two camera paths: expo-camera snapshots (default) or Vision Camera ML Kit (dev build). Formed by merging detection, depth, and foot services into one orchestrator.
SettingsScreen.js	~400	Audio/TTS/haptic toggles, detection precision, Bluetooth, fall detection, emotion test, logout, offline status.
EmergencyContactsScreen.js	~250	CRUD emergency contacts; syncs ApiService + EmergencyService; import from phone contacts.
EmotionDetectionScreen.js	~200	Modal front-camera emotion scan every 3 s; TTS announces dominant expression.

8. Components & Utilities

File	Purpose
RealTimeCameraView.js	Vision Camera + ML Kit frame processor; maps detections to app format; 15 FPS
ARCanvas.js	Expo-GL + Three.js wrapper for 3D AR overlays
Button.js / Card.js	Reusable themed UI components
LoadingScreen.js / ErrorBoundary.js	Loading states and crash recovery
faceApiPolyfill.js	window/document polyfill so face-api runs in Hermes RN
helpers.js / validators.js	formatDistance, debounce, coordinate validation
constants/config.js	API_BASE_URL, Google Maps key, detection/navigation constants
constants/theme.js	Dark theme colors, spacing, typography

9. Backend Server — Module by Module

9.1 Entry — server/src/index.js

Connects MongoDB, mounts routers under `/api/`, serves static ML models, exposes `GET /api/health`, initializes Socket.IO.

9.2 Routes

Router	Key Endpoints	Logic
auth.js	POST /register, /login	bcrypt hash, JWT sign (7 days)
users.js	GET/PUT /me, /preferences, emergency-contacts CRUD	requireAuth middleware on all routes
ai.js	POST /ocr, /detection/*, /voice-command, /emotion, /depth	Tesseract OCR; session logging for detections; voice audit log
navigation.js	POST /route, GET /status, POST /stop, /routes, /history	Navigation session + route persistence
emergency.js	POST /alert, GET /alerts, PUT /:id/resolve	SOS alert storage + Socket.IO emit
offline-maps.js	CRUD /offline-maps	Offline map region metadata
logs.js	GET/POST /logs, DELETE /clear	System event logging

9.3 Database Models (Mongoose)

User, UserProfile, UserPreferences, EmergencyContact, EmergencyAlert, NavigationSession, Route, DetectionSession, DetectedObject, VoiceCommand, OCRSession, SavedLocation, WearableDevice, OfflineMap, SystemLog — each maps to a MongoDB collection with schema validation.

9.4 Static Model Hosting

Run `node server/scripts/download-coco-ssd.js` once to populate weight shards. Server serves:

- `/models/coco-ssd/ssdlite_mobilenet_v2/` — ~18 MB SSDLite (primary)
- `/models/face-api/` — TinyFaceDetector + FaceExpressionNet weights

10. Data Flow Diagrams

10.1 AR Detection Frame Flow

```
Camera snapshot → ImageManipulator (320px JPEG) → decodeJpeg → TF tensor →
ObjectDetectionService.detectFromTensor → COCO-SSD → NMS → tracking → hazard score →
extractImageEdgeSignal (brightness rows) → DepthEstimationService.processFrame →
FootPlacementService.processFrame → UI overlays + TTS + SpatialAudio + server log
```

10.2 Voice Command Flow

```
Microphone → expo-speech-recognition → "Hey Sensei" wake word detected → Command mode
(listening) → normalizeInput + fuzzy match COMMAND_DEFINITIONS → dispatch action (navigate /
start AR / SOS / read text / ...) → TextToSpeechService feedback
```

10.3 Navigation Flow

```
User: "Navigate to [destination]" → VoiceCommandService → PinnedLocationService fuzzy lookup →
GoogleMapsService.getDirections(origin, dest) → NavigationService.startNavigation(route) →
LocationService.watchPosition → step advance → TTS turn instructions + SpatialAudio cues →
Optional: log session to server
```

11. Configuration & Deployment

File	Key Settings
src/constants/config.js	API_BASE_URL (LAN IP:3001), GOOGLE_MAPS_API_KEY
app.json	Camera/mic/location permissions; vision-camera plugin
eas.json	EAS build profiles (development, preview, production)
metro.config.js	TF.js + face-api transform allowlist
server/.env	MONGODB_URI, JWT_SECRET, PORT, GOOGLE_MAPS_API_KEY

Build requirements for full features:

- Object detection: works in Expo dev client after `npx expo run:android`
- Voice commands: requires native rebuild for expo-speech-recognition
- Real-time Vision Camera: requires dev build (not Expo Go)
- Server: MongoDB running + `node server/scripts/download-coco-ssd.js` + `npm start` in `server/`

12. Testing Strategy

- **Client Jest tests** (`__tests__`): ObjectDetection, Navigation, UserManagement, IntegratedModules
- **Server tests** (`server/tests/api.test.js`): Auth register/login via supertest
- **CI** (`.github/workflows/ci.yml`): Server tests on push/PR
- **Coverage target**: 70% (`jest.config.js`)

13. Learning Roadmap — What to Study

Topic	Why for SENSEI	Start Here in Codebase
React Native + Expo	App structure, navigation, native modules	App.js, ARScreen.js
TensorFlow.js	On-device ML inference	ObjectDetectionService.js
Computer Vision basics	SSD, NMS, bounding boxes, IoU	ObjectDetectionService detectFromTensor
Pinhole camera model	Distance from bbox height	_estimateDistance()
Graph algorithms	Indoor navigation	IndoorNavigationService Dijkstra
Google Maps API	Outdoor routing	GoogleMapsService.js
Speech recognition	Voice UX	VoiceCommandService.js
Levenshtein / fuzzy matching	Robust voice parsing	VoiceCommandService utilities
Node.js + Express + MongoDB	Backend API	server/src/
JWT authentication	Secure API access	server/src/middleware/auth.js
Three.js / WebGL	AR 3D rendering	ARService.js, ARCanvas.js
Accessibility UX	TTS, spatial audio, haptics	TextToSpeechService, SpatialAudioService

Recommended reading order for new developers:

1. README.md + QUICK_START_GUIDE.md
2. App.js → AppInitializer.js → one screen (HomeScreen)
3. ObjectDetectionService.js (core algorithm)
4. ARScreen.js (how services connect)
5. VoiceCommandService.js (voice OS)
6. NavigationScreen.js + GoogleMapsService.js
7. server/src/index.js + ai.js routes
8. docs/ folder (API.md, AR_INTEGRATION.md, TESTING.md)